Análisis Laboratorio 3 Programación Orientada a Objetos

1. Requisitos funcionales

a) Información que debe conservar

Clase	Información
Orden de servicio	Identificador, nombre del cliente, dirección, horas estimadas de trabajo, zona, estado, cantidad de visitas, jornada actual, indicador de registro, cobro inicial y cargos adicionales.
Instalación	Cantidad de puntos de instalación.
Instalación eléctrica	Valor de los materiales utilizados.
Instalación de paneles solares	Cantidad de paneles.
Mantenimiento de AC	Capacidad del equipo en BTU.
Limpieza de cisterna	Capacidad de la cisterna en litros.
Cuadrilla	Código, capacidad de horas, certificación en refrigeración y jornada abierta.
Jornada	Cuadrilla responsable, estado, órdenes asignadas y resultados de las visitas.
Resultado de visita	Orden atendida e indicador de si fue completada.

b) Operaciones y validaciones

Operación	Comportamiento	Condiciones de validez
Cotizar	Mostrar el costo y los días hábiles estimados.	Los datos deben cumplir las reglas de la categoría. No registra ni cobra la orden.
Registrar una orden	Guardarla y conservar el cobro inicial calculado.	ID único, datos válidos y orden no registrada previamente.
Cargar órdenes iniciales	Registrar órdenes de ejemplo sin cobro inicial.	ID único y orden no registrada previamente.
Registrar una cuadrilla	Incorporar un equipo de trabajo.	Código obligatorio y único; capacidad finita y positiva.
Crear una jornada	Asociar una jornada con una cuadrilla.	La cuadrilla debe existir y no tener otra jornada abierta.
Asignar una orden	Incorporarla a una jornada.	Jornada en preparación; orden registrada, pendiente, sin otra jornada y con visitas disponibles; capacidad suficiente y certificación cuando corresponda.
Retirar una orden	Eliminar su asignación actual.	Jornada en preparación y orden perteneciente a ella.
Iniciar una jornada	Iniciar una visita para cada orden asignada.	Jornada en preparación, no vacía y con todas sus órdenes habilitadas para iniciar visita.

Operación	Comportamiento	Condiciones de validez
Registrar un resultado	Indicar si la visita completó el servicio.	Jornada en curso; orden perteneciente a ella, en atención y sin resultado previo en esa jornada.
Cerrar una jornada	Finalizarla y liberar la cuadrilla.	Todas las órdenes tienen un resultado registrado. El cierre es automático.
Consultar registros	Mostrar órdenes, cuadrillas y jornadas.	No requiere condiciones adicionales.
Generar reportes	Mostrar órdenes por categoría y estado, ingresos y jornadas abiertas.	No requiere condiciones adicionales.
Seleccionar una opción del menú	Ejecutar la operación correspondiente.	La opción debe estar entre 1 y 12; de lo contrario, se informa que es inválida.

c) Validaciones de datos

Dato	Condición
ID, cliente y dirección	No deben ser nulos ni estar vacíos después de eliminar espacios exteriores.
Horas de trabajo	Número finito mayor que cero.
Zona	Metropolitana o periférica
Diagnóstico	Las horas estimadas no deben superar 1.5.

Dato	Condición
Puntos de instalación	Entero positivo.
Valor de materiales	Número mayor que 0
Capacidad de AC	Entre 9,000 y 60,000 BTU.
Zona del mantenimiento de AC	Únicamente metropolitana.
Cantidad de paneles	Entero positivo.
Capacidad de cisterna	Número mayor que cero.

2. **Decisiones que dependen según el tipo de servicio**

Clase	Costo	Días hábiles	Máximo de visitas
Diagnostico	Q150 metropolitana o Q225 periférica.	1 metropolitana o 2 periférica.	3
Instalacion	Tarifa base + 90 * (mayor de horas de trabajo y facturables por 0.75)	3 metropolitana o 5 periférica.	3
InstalacionElectrica	Costo de instalación + Q120 + (0.05 * valor de materiales).	Plazo de instalación + 2 días.	2

Clase	Costo	Días hábiles	Máximo de visitas
MantenimientoAC	100 + 80 + (110 * horas de trabajo)	2.	1
InstalacionPanelesSolares	Tarifa base + 90 *(maximo entre horas y puntos por 0.8) + (200 + cantidad de paneles)	Plazo de instalación + 5.	3
LimpiezaCisterna	Base de Q150 metropolitana o Q225 periférica + 80 * (máximo de horas y Litros/1000) + 50	Capacidad de la cisterna en litros divido entre mil	2

3. **Estados de una orden y una jornada**

   Orden:

Estado inicial	Operación o condición	Estado final	Efectos
Objeto recién creado	Constructor	PENDIENTE	Se inicializa sin registrar, sin visitas ni cobros.
PENDIENTE sin registrar	Registrar	PENDIENTE	Se marca como registrada y se guarda su cobro inicial.
PENDIENTE sin registrar	Registrar como dato inicial	PENDIENTE	Se marca como registrada sin cobro inicial.

Estado inicial	Operación o condición	Estado final	Efectos
PENDIENTE	Asignar o retirar de una jornada	PENDIENTE	Cambia su asociación con la jornada.
PENDIENTE	Iniciar visita	EN_ATENCION	Aumentan las visitas y se aplica recargo desde la segunda.
EN_ATENCION	Registrar visita completada	COMPLETADA	Se libera la asociación actual con la jornada.
EN_ATENCION	Registrar visita fallida con visitas disponibles	PENDIENTE	Puede asignarse a otra jornada.
EN_ATENCION	Registrar visita fallida sin visitas disponibles	CANCELADA	No permite nuevas visitas.

Jornada:

Estado inicial	Operación o condición	Estado final	Efectos
No existe	Crear jornada	PREPARACION	Se asocia con una cuadrilla.
PREPARACION	Asignar o retirar órdenes	PREPARACION	Cambia la planificación.
PREPARACION	Iniciar jornada	EN_CURSO	Inicia las visitas de todas sus órdenes.
EN_CURSO	Registrar un resultado y todavía faltan otros	EN_CURSO	Guarda el resultado.

Estado inicial	Operación o condición	Estado final	Efectos
EN_CURSO	Registrar el último resultado	CERRADA	Libera la cuadrilla.

4. Clases

Orden Servicio:

Atributo	Tipo	Propósito
id	String	Identificar la orden.
nombreCliente	String	Identificar al cliente.
direccion	String	Indicar el lugar de atención.
horasTrabajo	double	Conservar las horas estimadas.
zona	String	Determinar el área del servicio.
estado	String	Controlar su situación actual.
cantidadVisitas	int	Contar las visitas iniciadas.
jornada	Jornada	Referenciar la jornada actual.
registrada	boolean	Indicar si fue registrada.
montoCobrado	double	Conservar el cobro inicial.
cargosAdicionales	double	Acumular recargos por visitas.

Constructor o método	Visibilidad	Parámetros y que devuelve	Propósito
OrdenServicio(...)	Pública	String id, String nombreCliente, String direccion, double horasTrabajo, String zona	Validar e inicializar datos comunes.
getId(), getNombreCliente(), getDireccion(), getZona(), getEstado()	Pública	Sin parámetros → String	Consultar datos textuales.
getHorasTrabajo(), getMontoCobrado(),getCargosAdicionales()	Pública	Sin parámetros → double	Consultar horas y montos.
getCantidadVisitas()	Pública	Sin parámetros → int	Consultar visitas.
getJornada()	Pública	Sin parámetros → Jornada	Consultar la asignación actual.
estaRegistrada()	Pública	Sin parámetros → boolean	Consultar el registro.
setZona(...)	Pública	String zona → void	Validar y normalizar la zona.

Constructor o método	Visibilidad	Parámetros y que devuelve	Propósito
registrar()	Paquete	Sin parámetros → void	Registrar y guardar el costo inicial.
registrarSinCobro()	Paquete	Sin parámetros → void	Registrar sin cobro inicial.
asignarJornada(...)	Paquete	Jornada jornada → void	Asignar o liberar la jornada.
validarInicioVisita()	Paquete	Sin parámetros → void	Comprobar registro, estado y visitas disponibles.
iniciarVisita()	Paquete	Sin parámetros → void	Contar la visita, aplicar recargo y cambiar el estado.
registrarResultado(...)	Paquete	boolean completada → void	Resolver el estado posterior a la visita.
calculoServicio()	Pública, abstracta	Sin parámetros → double	Definir el costo.

Constructor o método	Visibilidad	Parámetros y que devuelve	Propósito
calculoTiempo()	Pública, abstracta	Sin parámetros → int	Definir los días hábiles.
getMaximoVisitas()	Pública, abstracta	Sin parámetros → int	Definir el límite de visitas.

Subclases servicio

Clase y responsabilidad	Atributos adicionales privados	Parámetros adicionales del constructor
Diagnostico: aplicar las reglas de diagnóstico.	Ninguno.	Ninguno.
Instalacion: manejar puntos y reglas generales de instalación.	Integer puntosInstalacion.	Integer puntosInstalacion.
InstalacionElectrica: incorporar materiales y reglas eléctricas.	double valorMateriales.	Puntos y double valorMateriales.
InstalacionPanelesSolares: incorporar paneles y reglas solares.	int cantidadPaneles.	Puntos e Integer cantidadPaneles.
MantenimientoAC: validar y calcular mantenimiento de aire acondicionado.	Integer capacidadBTU.	int capacidadBTU.
LimpiezaCisterna: calcular el servicio según capacidad.	double capacidadLitros.	double capacidadCisterna.

Clase	Método, visibilidad, resultado y propósito
Diagnostico	private static double validarHoras(double horasTrabajo): valida el máximo y devuelve las horas.
Instalacion	public Integer getPuntos(): consulta puntos. public void setPuntos(Integer puntos): valida y asigna. public double horasFacturables(): calcula las horas cobrables. public int calcularTarifaBase(): obtiene la tarifa según zona.
InstalacionElectrica	public double getMateriales(): consulta materiales. public void setMateriales(double materiales): valida y asigna su valor.
InstalacionPanelesSolares	public void setPaneles(int cantidad): valida y asigna la cantidad al atributo. Sobrescribe public double horasFacturables() para utilizar su factor particular.
MantenimientoAC	public void setBTU(int capacidad): valida y asigna BTU. private static String validarZona(String zona): valida y devuelve la zona normalizada. Sobrescribe public void setZona(String zona) para mantener la restricción metropolitana.
LimpiezaCisterna	public void setCapacidad(double capacidad): valida que sea finita y positiva y la asigna. public double getCapacidad(): consulta litros. public double horasFacturables(): calcula horas cobrables. public int calcularTarifaBase(): obtiene la tarifa según zona.

Clase Cuadrilla

Atributo privado	Tipo	Propósito
codigo	String	Identificar la cuadrilla.
capacidadHoras	double	Limitar las horas asignables.
certificadaRefrigeracion	boolean	Indicar habilitación para mantenimiento de AC.
jornadaAbierta	Jornada	Controlar la jornada actual.

Constructor o método	Visibilidad	Parámetros y su resultado	Propósito
Cuadrilla(...)	Pública	String codigo, double capacidadHoras, boolean certificadaRefrigeracion	Validar e inicializar el equipo.
getCodigo()	Pública	Sin parámetros → String	Consultar código.
getCapacidadHoras()	Pública	Sin parámetros → double	Consultar capacidad.
estaCertificadaRefrigeracion()	Pública	Sin parámetros → boolean	Consultar certificación.
getJornadaAbierta()	Pública	Sin parámetros → Jornada	Consultar jornada actual.
asociarJornada(...)	Paquete	Jornada jornada → void	Asociar una jornada si está disponible.

Constructor o método	Visibilidad	Parámetros y su resultado	Propósito
liberarJornada(...)	Paquete	Jornada jornada → void	Liberar la asociación correspondiente.

Clase Jornada

Atributo privado	Tipo	Propósito
cuadrilla	Cuadrilla	Referenciar al equipo responsable.
estado	String	Controlar preparación, ejecución y cierre.
ordenes	ArrayList<OrdenServicio>	Conservar órdenes asignadas.
resultados	ArrayList<ResultadoVisita>	Conservar resultados registrados.

Constructor o método	Visibilidad	Parámetros → resultado	Propósito
Jornada(...)	Pública	Cuadrilla cuadrilla	Validar la cuadrilla, crear listas y asociar la jornada.
getEstado()	Pública	Sin parámetros → String	Consultar estado.
getCuadrilla()	Pública	Sin parámetros → Cuadrilla	Consultar responsable.

Constructor o método	Visibilidad	Parámetros → resultado	Propósito
getOrdenes()	Pública	Sin parámetros → ArrayList<OrdenServicio>	Devolver una copia de la lista de órdenes.
getResultados()	Pública	Sin parámetros → ArrayList<ResultadoVisita>	Devolver una copia de la lista de resultados.
validarPreparacion()	Privada	Sin parámetros → void	Exigir estado de preparación.
calcularHorasAsignadas()	Pública	Sin parámetros → double	Sumar las horas estimadas de trabajo.
asignarOrden(...)	Pública	OrdenServicio orden → void	Validar y asignar.
retirarOrden(...)	Pública	OrdenServicio orden → void	Retirar durante la preparación.
iniciar()	Pública	Sin parámetros → void	Iniciar las visitas y cambiar el estado.
tieneResultado(...)	Privada	OrdenServicio orden → boolean	Detectar resultados duplicados.
registrarResultado(...)	Pública	OrdenServicio orden, boolean completada → void	Registrar el resultado y cerrar cuando corresponda.

Resultado visita

Miembro	Tipo, visibilidad y propósito
orden	private OrdenServicio: identifica la orden atendida.
completada	private boolean: indica si se completó.
ResultadoVisita(OrdenServicio orden, boolean completada)	Constructor público que asigna ambos datos.
getOrden()	Público → OrdenServicio: consulta la orden.
estaCompletada()	Público → boolean: consulta el resultado.
getDescripcion()	Público → String: devuelve COMPLETADA o VISITA_FALLIDA.

GestorServicios

Atributo	Tipo	Propósito
ordenes	ArrayList<OrdenServicio>	Guardar órdenes registradas.
cuadrillas	ArrayList<Cuadrilla>	Guardar cuadrillas.
jornadas	ArrayList<Jornada>	Guardar jornadas.

Método	Visibilidad	Parámetros y su resultado	Propósito
cotizar(...)	Pública	OrdenServicio servicio → void	Mostrar costo y días hábiles.

Método	Visibilidad	Parámetros y su resultado	Propósito
buscarOrden(...)	Pública	String id → OrdenServicio	Localizar una orden.
validarIdDisponible(...)	Privada	String id → void	Impedir identificadores repetidos.
registrarOrden(...)	Pública	OrdenServicio orden → void	Registrar y almacenar con cobro.
registrarOrdenInicial(...)	Pública	OrdenServicio orden → void	Registrar y almacenar sin cobro inicial.
buscarCuadrilla(...)	Pública	String codigo → Cuadrilla	Localizar una cuadrilla.
registrarCuadrilla(...)	Pública	Cuadrilla nueva → void	Validar unicidad y almacenar.
crearJornada(...)	Pública	String codigoCuadrilla → int	Crear la jornada y devolver su número.
buscarJornada(...)	Pública	int numero → Jornada	Buscar una jornada por número.
consultarOrdenes(), consultarCuadrillas(), consultarJornadas()	Pública	Sin parámetros → void	Mostrar registros.

Método	Visibilidad	Parámetros y su resultado	Propósito
reporteOrdenes()	Pública	Sin parámetros → void	Contar órdenes por categoría y estado.
reporteIngresos()	Pública	Sin parámetros → void	Mostrar ingresos generales y por categoría.
reporteJornadasAbiertas()	Pública	Sin parámetros → void	Mostrar jornadas sin cerrar y capacidad disponible.

App/Main

Miembro	Tipo, visibilidad y propósito
gestor	GestorServicios, visibilidad de paquete: administra los registros.
entrada	private static Scanner: recibe datos de consola.
main(String[] args)	Público, devuelve void y declara throws Exception: carga datos y ejecuta el menú. En el archivo compartido es un método de instancia.
DatosIniciales()	Público, sin parámetros, devuelve void: carga órdenes y cuadrillas de ejemplo.
crearOrden()	Público, sin parámetros, devuelve void: solicita datos, construye el servicio, cotiza y ofrece registrarlo.

5. **Relación de instalación eléctrica e instalación**

   La relación propuesta es de herencia, una instalación eléctrica es una especialización de una instalación. Comparte sus puntos de instalación, tarifa base y cálculo de horas facturables, además de los datos y operaciones heredados de OrdenServicio. La eléctrica agrega el valor de materiales, un recargo fijo de Q120, un cargo del 5 % sobre materiales, dos días hábiles adicionales y un máximo de dos visitas.

   La herencia permite reutilizar las reglas comunes mediante super y sobrescribir únicamente lo que cambia, lo que hace mucho más fácil en comparación de si se hubiera heredado directamente desde OrdenServicio, porque se tendría que hacer otra instalación con todos los atributos y agregarle los específicos de la eléctrica.

6. **Categorías nuevas**

Nueva categoría	Relación elegida	Justificación
InstalacionPanelesSolares	Hereda de Instalacion.	Comparte puntos y tarifa base; modifica horas facturables y agrega paneles.
LimpiezaCisterna	Hereda de OrdenServicio.	Comparte casi todo de una orden, pero calcula su servicio mediante litros.

Para agregar otra categoría con reglas propias se necesitaría:

Clase	Creación o modificación
Nueva clase de servicio	Definir atributos, validaciones y constructor. Implementar costo, días hábiles y máximo de visitas.
App	Agregar la opción de categoría, solicitar sus datos y construir el objeto correspondiente.

Clase	Creación o modificación
Cuadrilla	Ampliarla si aparece una certificación o capacidad que todavía no representa.
Jornada	Modificar la validación si la categoría necesita una regla particular de compatibilidad con la cuadrilla.
GestorServicios	Agregar la categoría a las listas utilizadas por los reportes de órdenes e ingresos.

¿Tendrían que cambiar las jornadas?

Para paneles y cisternas, no se necesita una validación de certificación adicional según las reglas implementadas. Sus horas estimadas se incorporan al control de capacidad existente. Para una categoría con requisitos nuevos de cuadrilla, sí habría que modificar la validación actual.

¿Tendrían que cambiar los reportes?

Con las nuevas clases actuales sí, porque los reportes contienen listas fijas de categorías. Al agregar un servicio, deben actualizarse tanto el reporte de órdenes como de ingresos. Entonces en ese caso se debería de agregar la opción de las nuevas categorías de servicio que se incorporarían al sistema para que pudieran ser mostradas en los registros.